

实验七 VGA 接口控制器实现

2023 年秋季学期

大约二十瓶颜料在桌子上整整齐齐地排成一溜儿。修拉拿了一支温森特见过的最小号的画笔，把笔尖在其中的一瓶颜料里蘸了一下，就着手在画布上以数学上的那种精确点起色点来。他平静地、毫不动情地工作着，点着，点着，点着。他笔直地握着画笔，只在颜料瓶里蘸一下，就往画布上点啊点的，点上成百上千细小的色点。

— 《渴望生活·梵高传》，欧文·斯通

VGA 接口是 IBM 制定的一种视频数据的传输标准，是电脑显示器最典型的接口。本实验的目的是学习 VGA 接口原理，学习 VGA 接口控制器的设计方法。

7.1 VGA 简介

7.1.1 VGA 接口的外观和引脚功能

VGA（Video Graphics Array）接口，即视频图形阵列。VGA 接口最初是用于连接 CRT 显示器的接口，CRT 显示器因为设计制造上的原因，只能接受模拟信号输入，这就需要显卡能输出模拟信号。关于模拟信号和数字信号的区别，请参考 ([Analog Signal](#)及[Digital Signal](#))。VGA 接口就是显卡上输出模拟信号的接口，在传统的 CRT 显示器中，使用的都是 VGA 接口，现在仍有不少液晶显示器或投影仪还支持 VGA 口。VGA 接口是 15 针/孔的梯形插头，分成 3 排，每排 5 个，如图 7-1 所示：

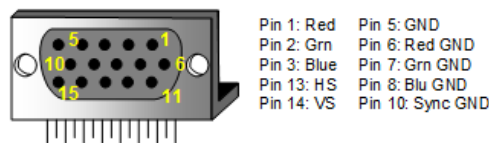


图 7-1: VGA 接口形状及信号示意图

VGA 接口的接口信号主要有 5 个：R（Red）、G（Green）、B（Blue）、HS（Horizontal Synchronization）和 VS（Vertical Synchronization），即红、绿、

蓝、水平同步和垂直同步（也称行同步和帧同步）。值得注意的是，VGA 信号是模拟信号，以电压值来表示颜色信号的强弱，所以只需要一根 R 信号线携带红色信号的强度信息。

7.1.2 VGA 的工作原理

图像的显示是以像素（点）为单位，显示器的分辨率是指屏幕每行有多少个像素及每帧有多少行，标准的 VGA 分辨率是 640×480 ，也有更高的分辨率，如 1024×768 、 1280×1024 、 1920×1200 等。从人眼的视觉效果考虑，屏幕刷新的频率（每秒钟显示的帧数）应该大于 24，这样屏幕看起来才不会闪烁，VGA 显示器一般的刷新频率是 60HZ。

每一帧图像的显示都是从屏幕的左上角开始一行一行进行的，行同步信号是一个负脉冲，行同步信号有效后，由 RGB 端送出当前行显示的各像素点的 RGB 电压值，当一帧显示结束后，由帧同步信号送出一个负脉冲，重新开始从屏幕的左上端开始显示下一帧图像，如图 7-2 所示。

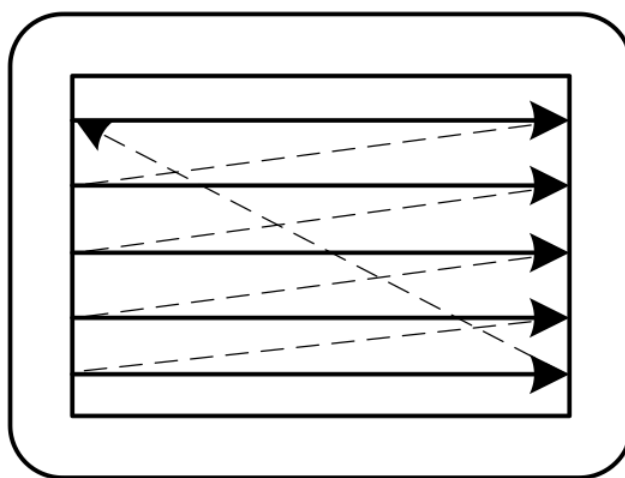


图 7-2: 显示器扫描示意图

RGB 端并不是所有时间都在传送像素信息，由于 CRT 的电子束从上一行的行尾到下一行的行头需要时间，从屏幕的右下角回到左上角开始下一帧也需要时间，这时 RGB 送的电压值为 0（黑色），这些时间称为电子束的行消隐时间和场消隐时间，行消隐时间以像素为单位，帧消隐时间以行为单位。VGA 行扫描、场扫描时序示意图如图 7-3 所示：

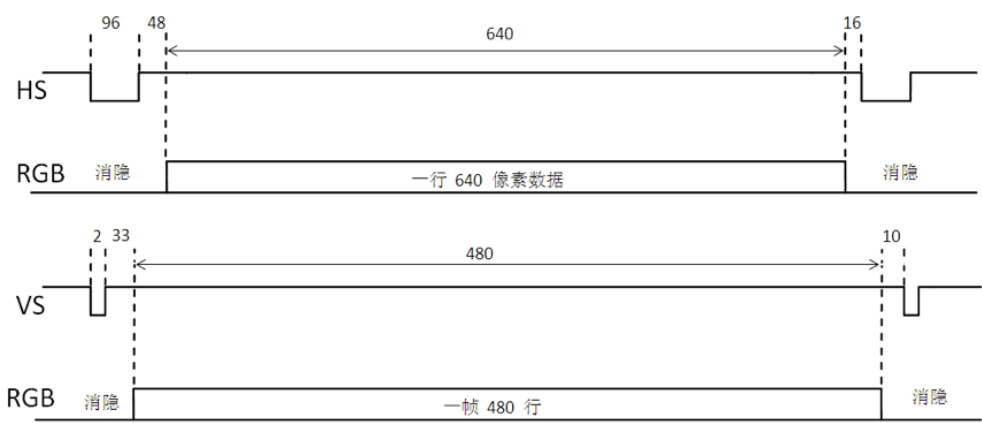


图 7-3: VGA 行扫描、场扫描时序示意图

由图 7-3 可知，在标准的 640×480 的 VGA 上有效地显示一行信号需要 $96+48+640+16=800$ 个像素点的时间，其中行同步负脉冲宽度为 96 个像素点时间，行消隐后沿需要 48 个像素点时间，然后每行显示 640 个像素点，最后行消隐前沿需要 16 个像素点的时间。所以一行中显示像素的时间为 640 个像素点时间，一行消隐时间为 160 个像素点时间。

在标准的 640×480 的 VGA 上有效显示一帧图像需要 $2+33+480+10=525$ 行时间，其中场同步负脉冲宽度为 2 个行显示时间，场消隐后沿需要 33 个行显示时间，然后每场显示 480 行，场消隐前沿需要 10 个行显示时间，一帧显示时间为 525 行显示时间，一帧消隐时间为 45 行显示时间。

因此，在 640×480 的 VGA 上的一幅图像需要 $525 \times 800 = 420k$ 个像素点的时间。而每秒扫描 60 帧共需要约 25M 个像素点的时间。

7.2 VGA 显示的实现

7.2.1 Nexys A7-100T 开发板上的 VGA 接口

Nexys A7-100T 开发板上使用不同阻值的电阻来将 FPGA 数字信号转换到 VGA 模拟信号，对于每种颜色，FPGA 提供 4bit 的亮度信息，具体连接方式如图 7-4 所示。

对于红色信号，开发板提供 4bit 的 RED[3:0]，当所有比特均为 0 时，合成的模拟红色信号电压为 0，亮度最低。如果所有比特都为 1 时，合成电压为

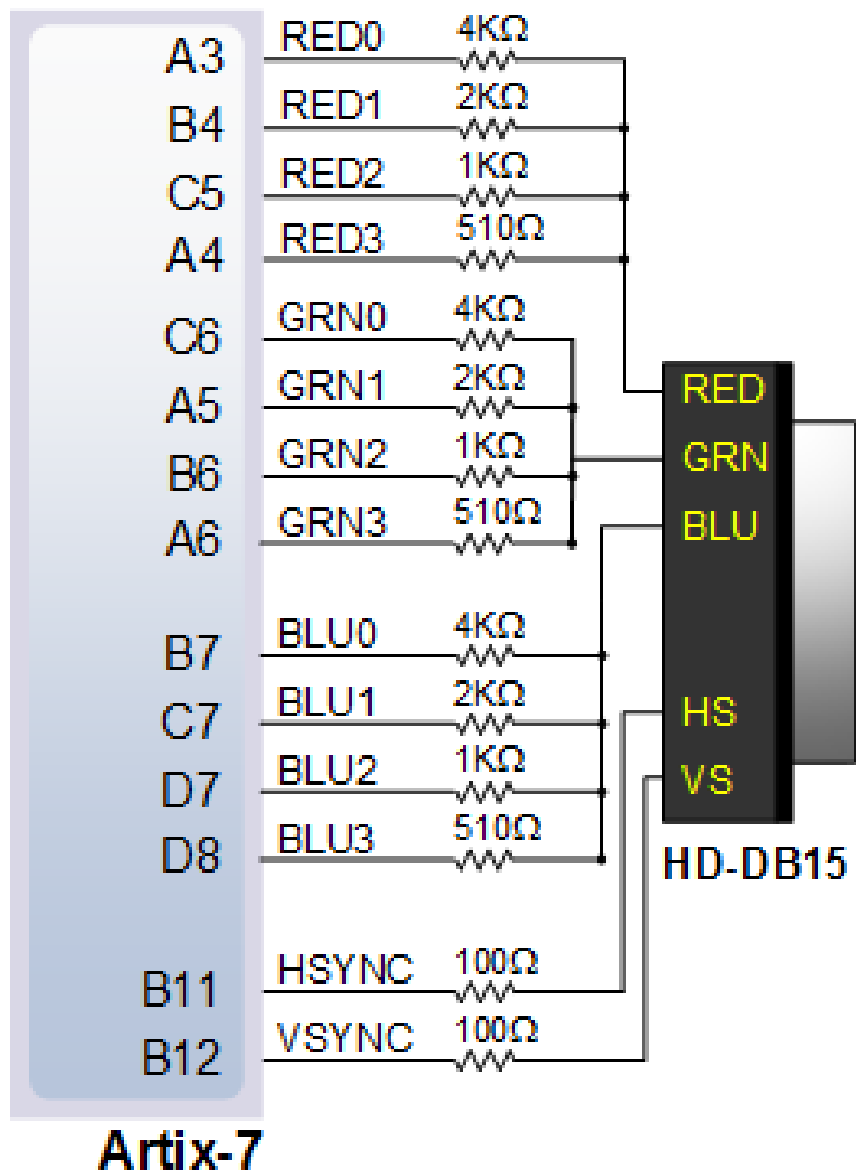


图 7-4: Nexys A7-100T 的 VGA 连接示意图

0.7V（VGA 信号线自身内阻为 75 欧姆），红色亮度最高。不同比特位的权重通过连接的电阻大小来确定，因此，通过 4bit 的 RED[3:0] 可以产生 16 种不同的红色亮度。这样三种颜色组合，可以形成 4096 种颜色，已经可以实现不错的显示效果了。

VGA 信号首先需要有一个时钟驱动，我们这里使用 25MHz 的时钟来驱动 VGA_CLK。每个时钟周期扫过一个像素点，因此在 640×480 的分辨率下，我们需要 $800 \times 525 = 420,000$ 个时钟周期才能扫描完一帧（此处考虑了消隐的时间）。在 25MHz 的时钟周期下总时长为 16.8 毫秒，对应约每秒约 60 帧。

我们可以使用实验 4 中提到的简单的分频器来从 100MHz 的时钟来产生所需的 VGA_CLK。

```
1 clkgen #(25000000) my_vgaclk(CLK100MHZ,SW[0],1'b1,VGA_CLK);
```

生成时钟的另一种方式是通过 Vivado 的 Clocking Wizard IP 核来实现。如图 7-5 所示，在 IP Catalog 中搜索 Clocking Wizard，双击对应 IP 后进行 Clocking Wizard 设置。

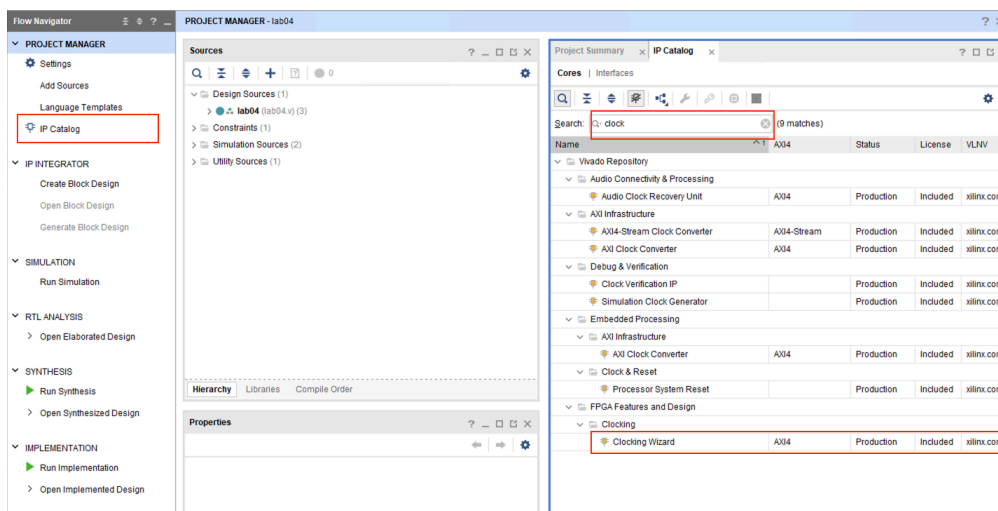


图 7-5: IP 目录中的 Clocking Wizard

Clocking Wizard 是预先开发好的分频模块，相比简单的计数器分频可以产生更加稳定的高质量时钟信号。图 7-6 是 Clocking Wizard 的设置界面。其中，模块名为 clk_wiz_0，输入时钟为 clk_in1，这里我们可以设置为缺省的系统 100MHz 时钟，如图 7-7 所示。输出可以根据需要产生一个或多个时钟，每个时钟的频率可以自行设置。这里我们设置 clk_out1 为 25MHz，如图 7-8 所示。Clocking Wizard 可以生成非整数倍的分频时钟，相对灵活度较高，当系统无法精确生成所需时钟频率时，会提示频率误差。该模块还包括 Reset 和 Locked 等控制信号。

在完成配置后点击 **OK**，随后点击新对话框中的 **Generate** 就可以生成对应的时钟模块了。

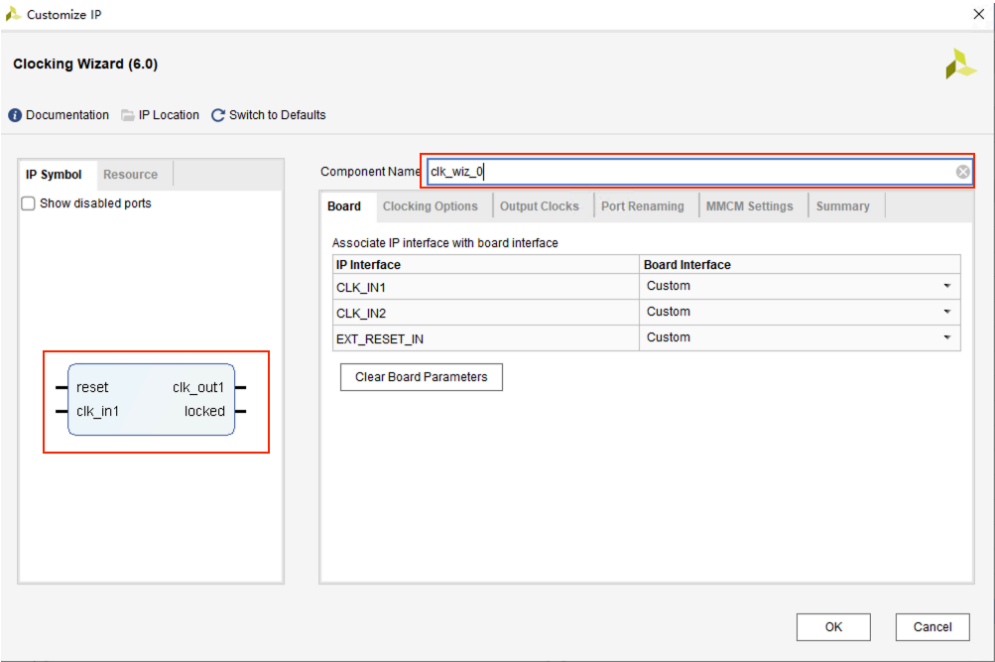


图 7-6: Clocking Wizard 的设置界面

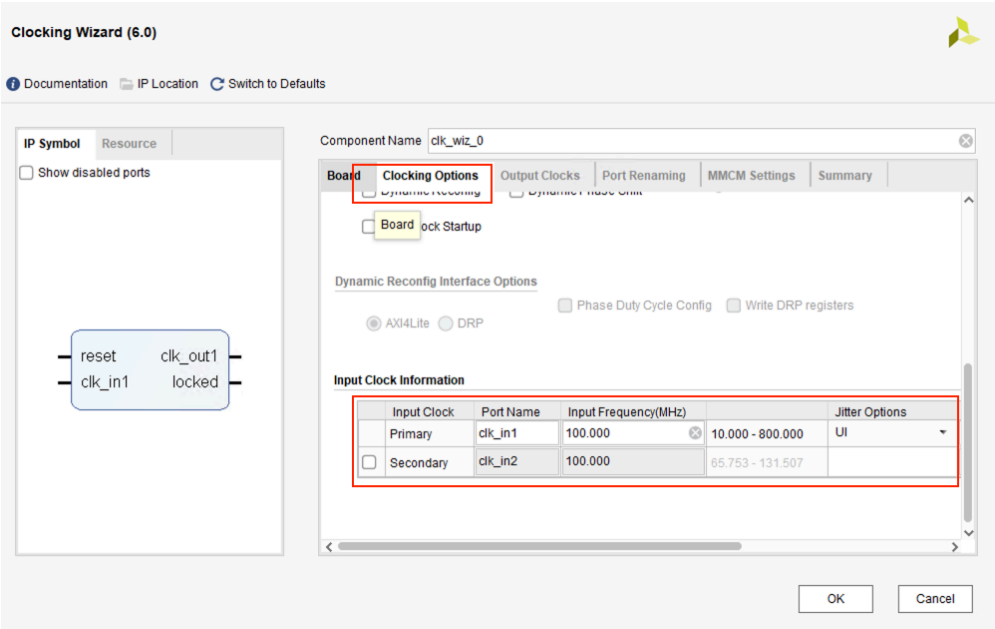


图 7-7: 缺省输入时钟频率

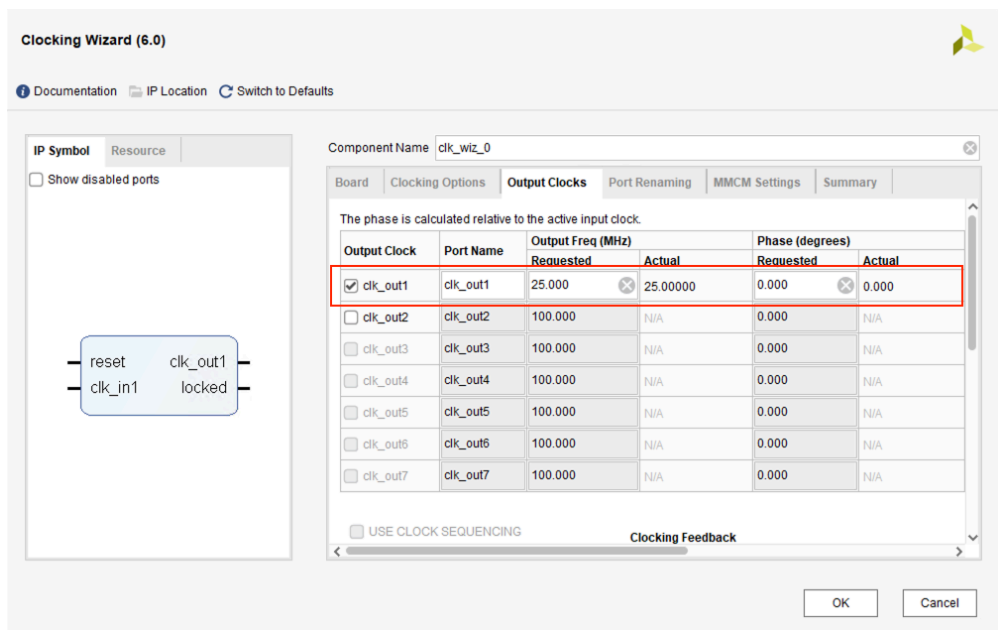


图 7-8: 输出时钟频率设置

生成完毕后，可以在工程中用如下方式来调用该模块。

```
1 clk_wiz_0 myvgaclk(.clk_in1(CLK100MHZ),.reset(SW[0]),
2                  .locked(LED[0]),.clk_out1(vga_clk));
```

在该时钟的驱动下我们需要生成各类驱动信号。其中 VGA 同步信号 VGA_SYNC_N 可以长期置零。其他信号可以参考表 7-1 来实现。

表 7-1: VGA 参考代码

```
1 module vga_ctrl(
2     input          pclk,          //25MHz 时钟
3     input          reset,         //置位
4     input  [11:0]  vga_data,      //上层模块提供的VGA颜色数据
5     output [9:0]   h_addr,        //提供给上层模块的当前扫描像素点坐标
6     output [9:0]   v_addr,
7     output         hsync,         //行同步和列同步信号
8     output         vsync,
9     output         valid,         //消隐信号
10    output [3:0]   vga_r,         //红绿蓝颜色信号
11    output [3:0]   vga_g,
12    output [3:0]   vga_b
```

```
13     );
14
15     //640x480 分辨率下的VGA参数设置
16     parameter    h_frontporch = 96;
17     parameter    h_active = 144;
18     parameter    h_backporch = 784;
19     parameter    h_total = 800;
20
21     parameter    v_frontporch = 2;
22     parameter    v_active = 35;
23     parameter    v_backporch = 515;
24     parameter    v_total = 525;
25
26     // 像素计数值
27     reg [9:0]    x_cnt;
28     reg [9:0]    y_cnt;
29     wire        h_valid;
30     wire        v_valid;
31
32     always @(posedge reset or posedge pclk) //行像素计数
33         if (reset == 1'b1)
34             x_cnt <= 1;
35         else
36             begin
37                 if (x_cnt == h_total)
38                     x_cnt <= 1;
39                 else
40                     x_cnt <= x_cnt + 10'd1;
41             end
42
43     always @(posedge pclk) //列像素计数
44         if (reset == 1'b1)
45             y_cnt <= 1;
46         else
47             begin
48                 if (y_cnt == v_total & x_cnt == h_total)
49                     y_cnt <= 1;
```



```

50         else if (x_cnt == h_total)
51             y_cnt <= y_cnt + 10'd1;
52         end
53         // 生成同步信号
54         assign hsync = (x_cnt > h_frontporch);
55         assign vsync = (y_cnt > v_frontporch);
56         // 生成消隐信号
57         assign h_valid = (x_cnt > h_active) & (x_cnt <= h_backporch);
58         assign v_valid = (y_cnt > v_active) & (y_cnt <= v_backporch);
59         assign valid = h_valid & v_valid;
60         // 计算当前有效像素坐标
61         assign h_addr = h_valid ? (x_cnt - 10'd145) : {10{1'b0}};
62         assign v_addr = v_valid ? (y_cnt - 10'd36) : {10{1'b0}};
63         // 设置输出的颜色值
64         assign vga_r = vga_data[11:8];
65         assign vga_g = vga_data[7:4];
66         assign vga_b = vga_data[3:0];
67     endmodule

```

此代码对外提供了 VGA 控制信号，利用对时钟进行计数来判断当前是在扫描第几行的第几个像素，并确定是否要消隐。代码输出的红 R、绿 G、蓝 B 三种颜色分别是以 vga_r, vga_g, vga_b 三个 4 位的二进制信号表示的，这三组 4 位数字信号将被传送到对应引脚，转换成模拟信号，经 VGA 接口送入显示器中。

该控制器的特点是可以方便地实现上层系统对显示内容的控制。例如，如果在模块调用时设置 vga_data 为常数 12'hF00，就可以直接显示全屏红色。上层系统也可以根据当前扫描的像素坐标，选择合适的颜色给不同的像素设置不同的 vga_data。更重要的是，上层系统可以分配一块显示存储，利用 v_addr, h_addr 来索引该显存，每次扫描到特定像素点时，按照显存的值来设置 vga_data。这样，其他应用就可以直接对显存进行操作，显存改变自动对应到 VGA 的显示上，而不用担心 VGA 扫描的具体过程了。

显存设计：如上所示，VGA 的每个像素需要 12 个 bit，对于 640 × 480 个像素就需要 3.686Mbit。我们之前提到过 Nexys A7-100T 上如果使用 LUT 中的资源实现 Distributed RAM，最大容量只有 1.188Mbit，不够存放显存。因此，我们需要使用大容量的 Block RAM 来实现显存，Nexys A7-100T 上有 4.86Mbit

的 Block RAM，可以实现 VGA 显存存储。

7.3 实验验收内容

7.3.1 上板验收

利用上述控制器，在显示器上显示一张静态图片。课程网站上提供了缺省图片的 coe 文件，也提供了图片到 coe 的 MATLAB 转换代码。

对于显存实现，可以采用 IP 核生成单口 RAM，每个单元 12bit，大小为 327,680 单元的存储（ 640×512 ）。用 h_addr 的全部 10 位和 v_addr 的低 9 位合成 19 位地址来索引显存。为方便寻址，我们给行 v_addr 分配了 512 行的空间。这样，可以不用对地址进行复杂的转换。此处只需要分配 327680 个连续的存储单元，不需要考虑 h_addr 大于 640 的情况。注意这里的像素数据其实是按列连续存放的。

拓展要求：

显示一张自定义的图片，自行完成图片格式到 coe 文件的转换。

7.3.2 在线测试

选做 确定有限状态自动机

选做 签到题